Studio Nx

JS 04 — loginUrl()

2026-10-12

JS 04 — `loginUrl(next)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	03 `safeNextPath()` · 05 `parseLoginHash()`

1. 하는 일

로그인 모달로 가는 주소를 만듭니다. 돌아올 경로가 있으면 `next`로 붙입니다.

`loginUrl('/myPage.html')` → `home.html#login?next=%2FmyPage.html`

2. 언제 쓰나

로그인 화면은 따로 없고 **`home.html`의 모달**입니다 (2026-10-02 결정). 그래서 "로그인하러 가기"는 항상 이 주소로 갑니다.

상황	부르는 곳
로그인이 필요한 페이지에 로그인하지 않고 들어옴	`requireLogin()`(07)이 `loginUrl(location.pathname + location.search)`로 부릅니다
회원가입을 마친 뒤 로그인하러 감	`join.js`(P1-07)가 `loginUrl()`로 부릅니다

주소의 `#` 뒷부분 (해시)

부분	쓰임
`#login`	`home.js`가 이것을 보고 로그인 모달을 자동으로 엽니다
`?next=…`	로그인에 성공한 뒤 돌아갈 경로입니다. `#` 뒤에 있으므로 **서버로 보내지지 않고** 브라우저 안에서만 쓰입니다

보안: 주소에 `next`가 보여도 괜찮은가?

한 줄 요약: 주소에 붙는 것은 "어디로 가려 했는지"를 적은 쪽지뿐이고, "열쇠"(토큰, 비밀번호)는 절대 주소에 붙지 않습니다.

비유: 건물 출입

건물	Studio Nx
3층 사무실에 가려는데 출입증이 없습니다	로그인하지 않고 `/myPage.html`에 들어옵니다
안내 데스크로 보내면서 "3층에 가려던 분"이라는 **쪽지**를 줍니다	`home.html#login?next=/myPage.html`로 보냅니다
안내 데스크에서 신분을 확인하고 **출입증**을 받습니다	로그인해서 토큰을 받습니다
쪽지를 보고 3층으로 안내합니다	`next`를 읽고 `/myPage.html`로 이동합니다

쪽지(`next`)는 누가 봐도 괜찮습니다. 가려던 층만 적혀 있기 때문입니다. 출입증(토큰)은 주머니(메모리, HttpOnly 쿠키)에만 넣고, 쪽지에는 절대 적지 않습니다.

순서와 주소창

단계	주소창
① 로그인하지 않고 마이페이지에 들어옴	`/myPage.html`
② 로그인하러 보냄: **이때** `next`가 붙습니다 (로그인 전)	`home.html#login?next=%2FmyPage.html`
③ 모달에 이메일과 비밀번호를 입력해서 성공	(그대로)
④ `next`를 검사하고 원래 페이지로 이동	`/myPage.html` (`next`는 사라집니다)

걱정과 답

걱정	답
주소에 비밀 정보가 남나요?	아닙니다. 남는 것은 경로뿐이고, 토큰은 메모리와 쿠키에만 있습니다
비밀번호가 주소에 붙나요?	아닙니다. 로그인은 `POST` 본문으로 보냅니다 (주소가 아니라 봉투 안에 넣는 셈입니다)
서버나 다른 사이트로 새어 나가나요?	아닙니다. `#` 뒷부분은 브라우저 안에만 있고 서버로 가지 않습니다
누가 `next`를 가짜 사이트로 바꾸면요?	`safeNextPath`가 이 사이트의 경로만 통과시킵니다 (03)
`next`에 스크립트를 넣으면요?	`javascript:` 같은 값은 거부합니다. 그리고 `next`는 **이동에만** 쓰고 화면에 HTML로 넣지 않습니다
`projectId=3`이 보이면요?	번호는 비밀이 아닙니다. 남의 프로젝트는 서버가 권한으로 막습니다 (P1-13)

구현할 때 지킬 것 (P1-07 로그인 폼)

1. `next`는 `location.href = next`처럼 **이동에만** 씁니다. `innerHTML`에 넣지 않습니다.
2. 로그인 폼은 JS로 기본 제출을 막고 `apiFetch`로 보냅니다. 폼에 `method="post"`가 있으므로 JS가 실패해도 비밀번호가 주소에 붙지 않습니다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 쓰지 않습니다.** `safeNextPath`와 `LOGIN_PAGE`는 같은 파일 안에 있습니다.

함수

```
export function loginUrl(next) {
  const path = safeNextPath(next);
  return path ? `${LOGIN_PAGE}#login?next=${encodeURIComponent(path)}` : `${LOGIN_PAGE}#login`;
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`encodeURIComponent(path)`	주소에서 특별한 뜻을 가진 글자를 `%XX`로 바꿉니다. `/` → `%2F`, `?` → `%3F`, `=` → `%3D`, `&` → `%26`
`` `${LOGIN_PAGE}#login?next=${…}` ``	**템플릿 문자열**입니다. `${ }` 안의 값을 끼워 넣은 문자열을 만듭니다
`path ? A : B`	안전한 경로가 있으면 A(`next`를 붙임), 없으면 B(`#login`만)입니다

4. 흐름

1. `safeNextPath`로 경로를 검사합니다 (03).
2. 안전하면 인코딩해서 `#login?next=` 뒤에 붙입니다.
3. 경로가 없거나 안전하지 않으면 `home.html#login`만 돌려줍니다.

5. 왜 이렇게

선택	이유
`encodeURIComponent`	경로 안의 `?`, `&`, `=`가 바깥쪽 `?next=`의 구분자로 잘못 읽히지 않게 합니다. 예를 들어 `/myProject.html?q=a&page=2`의 `&page=2`가 `next` 밖으로 새지 않습니다
주소를 만들 때도 `safeNextPath`로 검사	지금은 이 프로젝트의 코드만 부르지만, 잘못된 값이 들어와도 위험한 주소를 만들지 않습니다 (읽을 때도 한 번 더 검사합니다)
`LOGIN_PAGE` 상수	로그인 페이지가 바뀌어도 한 곳만 고치면 됩니다

6. 시험

입력	결과
`/myPage.html`	`home.html#login?next=%2FmyPage.html`
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html?q=a&page=2`	`…next=%2FmyProject.html%3Fq%3Da%26page%3D2`이고, 다시 읽으면 원래 경로 그대로입니다
없음, `''`	`home.html#login`
`https://evil.com`, `//evil.com`	`home.html#login` (`next`를 붙이지 않습니다)